the application might want to enforce a rule that the user stays logged in only as long as they're active; when they log out or some fixed time after they last use the application, their session should be terminated.

You can sometimes achieve this effect by expiring the cookie holding the session ID. But this is open to end-user abuse. Worse, it's hard to synchronize the expiry of a cookie on the browser with the tidying up of the session data on the server.

We therefore suggest you expire sessions by simply removing their server-side session data. Should a browser request subsequently arrive containing a session ID for data that's been deleted, the application will receive no session data; the session will effectively not be there.

Implementing this expiration depends on the storage mechanism being used.

For Active Record--based session storage, use the `updated_at` columns in the `sessions` table. You can delete all sessions that haven't been modified in the last hour (ignoring daylight saving time changes) by having your sweeper task issue SQL such as this:

```
delete from sessions
 where now() - updated_at > 3600;
```

For DRb-based solutions, expiry takes place within the DRb server process. You'll probably want to record timestamps alongside the entries in the session data hash. You can run a separate thread (or even a separate process) that periodically deletes the entries in this hash.

In all cases, your application can help this process by calling `reset_session` to delete sessions when they're no longer needed (for example, when a user logs out).

Flash: Communicating Between Actions